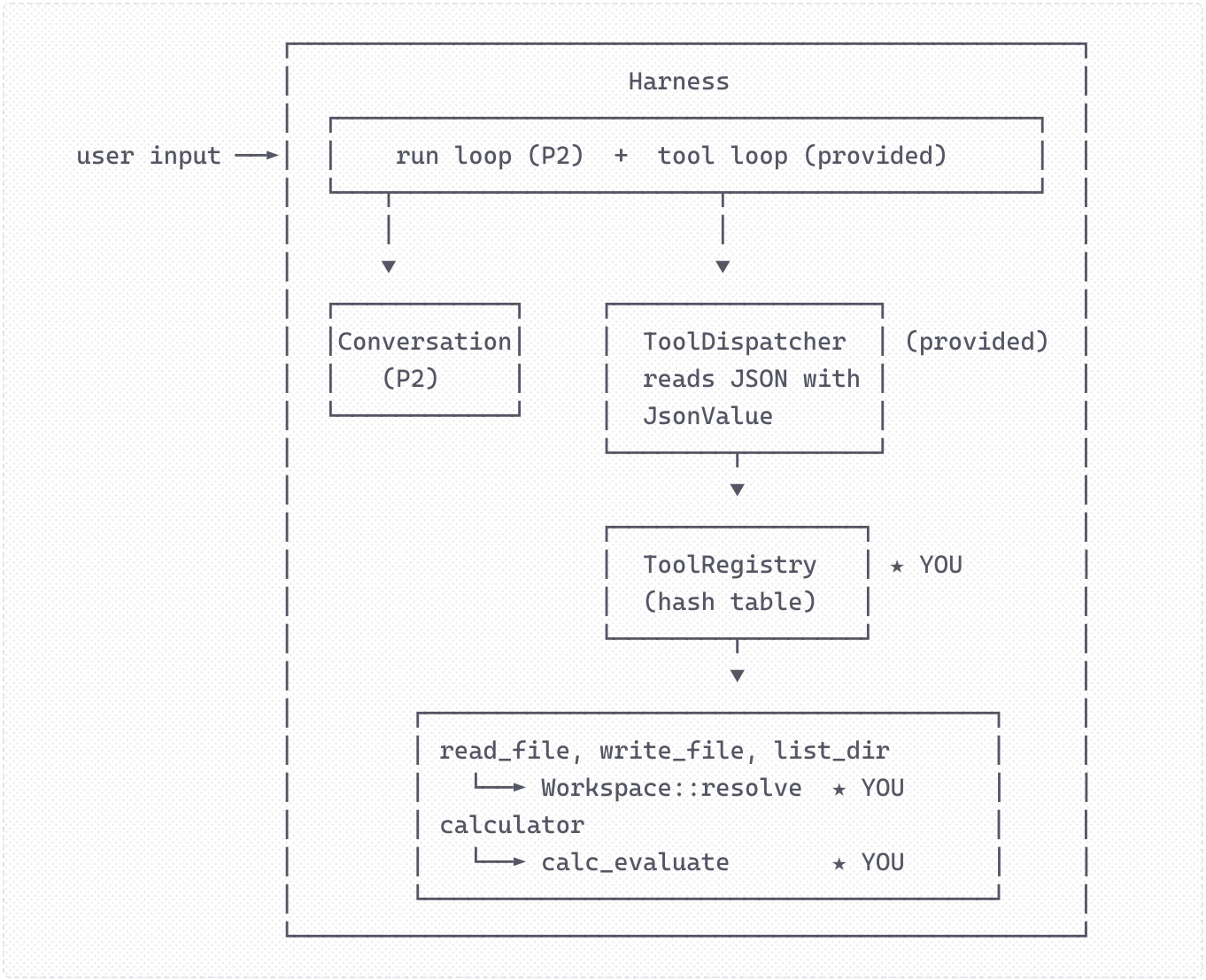
ECE 309: Project 3 — Tool Use and the Tool Registry

Weight: 20% of project grade | **Duration:** Weeks 7–8 (2 weeks) | **Deadline:** 10/22/2026
Expected effort: about 20–28 hours in total. There is a suggested schedule in §7.

1. Project Overview & Learning Goals

In Project 2 your harness could hold a conversation, but the model could only *talk*. In Project 3 the model gets to *act*. It can ask the harness to read a file, write a file, list a directory, or do arithmetic. The harness finds the request in the model's reply, runs the tool, adds the result to the conversation, and asks the model again — **without** waiting for a new user turn. Control returns to the user only when the model replies with plain text. This loop is at the center of every LLM agent.



Learning goals:

- Implement a **hash table** from scratch, with collision handling and resizing, and measure how the load factor and the hash function affect it. This is the central exercise of P3.
- Use **ownership** correctly. The registry owns its tools through `std::unique_ptr`, and the pointers it hands out must stay valid.
- Use a **stack** to evaluate arithmetic, the classic two-stack algorithm.
- Treat errors as **values**, not exceptions, across an interface boundary, and explain why.
- Treat input from the model as untrusted: keep file paths inside a sandbox folder.

What's provided vs. what's yours. As in P2, the "plumbing" is given to you as working code. You implement **three** things:

#	You implement	Course topic	Points
1	`ToolRegistry` — a hash table from tool name to `Tool`	hash tables, linked lists / arrays	45
2	`Workspace::resolve` — a short path-safety check	strings, `std::filesystem::path`	15
3	`calc_evaluate` — the calculator's evaluation step	stacks	10

Everything else is provided, including:

- the JSON reader/writer `JsonValue` (Appendix B explains JSON)
- the calculator's tokenizer, which turns `"(2+3)*4"` into a list of tokens
- the three file tools and the symbolic-link check
- the dispatcher, the extended `Harness` loop, transcripts, and `main.cpp`

You write no parsers. You should *read* the provided code that calls your functions (listed in §4.7), so you can see how they are used.

> **Platform requirement.** Just like previous projects, build and test on Linux: the department Linux machines, WSL2 on Windows, or a Linux VM. The grader runs on Linux with AddressSanitizer. Native Windows toolchains (MSVC/MinGW) are not supported for this project. macOS mostly works, but leak detection is unavailable there, so run your final check on Linux.

---

2. The Big Picture: Life of One Tool Call

Before reading the class descriptions, follow one tool call from start to finish. Every function you write appears in this story. The code is simplified from the provided `tool_dispatcher.cpp` and

`file_tools.cpp`.

Step 1: The model asks for a tool. The model's reply is just text. A tool call is a small JSON object between two tags (Appendix B explains JSON):

```
Sure, let me look.<|tool_call|>{"id":"c1","name":"read_file","arguments":
{"path":"notes/todo.txt"}}<|/tool_call|>
```

Step 2: The dispatcher reads it (provided). It cuts out the text between the tags and calls `JsonValue::parse` on it, which gives an object with members `id`, `name`, and `arguments`. You never write this code.

Step 3: The registry finds the tool (your hash table).

```
const JsonValue* name = call.find("name");          // provided JsonValue
Tool* tool = registry.find(name->as_string());      // ★ your
ToolRegistry::find
if (tool == nullptr)
    return error_result("unknown tool: " + name->as_string());
```

Step 4: The tool runs (polymorphism). The dispatcher doesn't know which kind of tool it has; it just calls `invoke`. For `read_file`, the provided code does:

```
const JsonValue* p = args.find("path");
fs::path real;
std::string why;
if (!ctx.workspace.resolve(p->as_string(), &real, &why))   // ★ your
Workspace::resolve
    return ToolResult::failure(why);
if (!ctx.workspace.check_symlinks(real, &why))             // provided
    return ToolResult::failure(why);
// ... read the file ...
return ToolResult::success(contents);
```

For `calculator`, the provided tool calls the provided `calc_tokenize`, then **your** `calc_evaluate`.

Step 5: The result goes back into the conversation (provided). The dispatcher writes a `Role::Tool` message, again as JSON on a single line:

```
<|tool_result|>{"id":"c1","ok":true,"content":"- buy milk\n- write hash
table\n- finish ECE 309 project 3\n"}<|/tool_result|>
```

Step 6: Loop. The harness calls the model again. If the new reply contains more tool calls, steps 1–5 repeat. If it doesn't, control returns to the user.

In short: **the provided code reads and writes the text; your registry finds the tool; your workspace keeps file tools inside their folder; and your evaluator does the arithmetic.**

---

3. Program Behaviors & Requirements

Example CLI interaction (`scripts/tools_demo.script`):

```
$ ./build/miniharness --script scripts/tools_demo.script
you> show my todo
assistant> Sure, let me look at your todo list.<|tool_call|>
{"id":"c1","name":"read_file","arguments":{"path":"notes/todo.txt"}}
<|/tool_call|>
[tool read_file c1 -> ok] - buy milk\n- write hash table\n- finish ECE 309
project 3\n
assistant> You have three items: buy milk, write the hash table, and finish
project 3.
you> what is (12+30)*2/7
assistant> Let me work that out.<|tool_call|>
{"id":"c2","name":"calculator","arguments":{"expression":"(12 + 30) * 2 / 7"}}
<|/tool_call|>
[tool calculator c2 -> ok] 12
assistant> (12 + 30) * 2 / 7 = 12.
you> save a summary
assistant> Saving a note and listing the folder.<|tool_call|>
{"id":"c3","name":"write_file","arguments":
{"path":"notes/summary.txt","content":"3 todo items\n"}}<|/tool_call|>
<|tool_call|>{"id":"c4","name":"list_dir","arguments":{"path":"notes"}}
<|/tool_call|>
[tool write_file c3 -> ok] wrote 13 bytes to notes/summary.txt
[tool list_dir c4 -> ok] summary.txt\t13\ntodo.txt\t57
assistant> Done: notes/summary.txt is saved next to todo.txt.
you> bye
assistant> Goodbye!
[conversation ended: Stop sentinel detected]
```

After each round of tool results, the model speaks again (`assistant>`) *before* the next `you>` prompt.

Required behaviors. The provided code implements these, but they only work if your three pieces are correct. The provided `test_loop` tests and the e2e golden sessions check them,

and the integration score comes from those.

1. **Tool loop.** After every assistant reply that contains at least one tool call, the harness runs every call in order, appends the results, and calls the model again with no user turn in between. A reply with no tool calls hands control back to the user.
2. **Step limit.** `--max-tool-steps N` (default 8) caps the tool rounds per user turn. When the cap is hit, the harness prints `[tool step limit reached; returning to user]` and returns to the prompt.
3. **Sentinel still wins.** If a reply contains `<|end_conversation|>`, the conversation ends, and tool calls in that reply are **not** executed.
4. **Malformed calls are data.** Bad JSON, a missing `name`, an unknown tool, or a missing closing tag each become an `"ok":false` result the model can read. None of them may crash the harness.
5. **Four built-in tools:**

Tool	Arguments	Behavior	Who writes it
`read_file`	`path`, optional `max_bytes`	file contents, up to 64 KiB	provided (uses your `resolve`)
`write_file`	`path`, `content`, optional `append`	creates parent folders; returns `wrote N bytes to PATH`	provided (uses your `resolve`)
`list_dir`	optional `path` (default `"."`), optional `recursive`	sorted entries: `name/` for folders, `name<TAB>size` for files	provided (uses your `resolve`)
`calculator`	`expression`	`+ - * /`, parentheses, decimals, negative numbers	provided (uses your `calc_evaluate`)

6. **Workspace containment.** No file tool may touch anything outside the workspace folder (`--workspace DIR`, default `./workspace`).
7. **Introspection.** `--tools` prints every registered tool's name, description, and parameters, sorted by name, then exits. It uses your `ToolRegistry::for_each`.
8. **P2 still works.** `--no-tools` runs exactly the P2 program, and every P2 test must still pass (§4.8).

Command-line flags (parsed by the provided `main.cpp`): `--script FILE`, `--replay FILE`, `--save FILE`, `--max-turns N`, `--max-tool-steps N`, `--workspace DIR`, `--tools`, `--no-tools`.

---

4. Architecture & Class Design

For the three pieces marked **you implement**, the **public** declarations are fixed: do not change them. You may add private members and helper functions to `ToolRegistry`, and `static` helper functions inside your `.cpp` files.

Class / file	Status
`Message` (adds `Role::Tool`)	Provided — replaces your P2 `message.h`
`Conversation`, `SentinelScanner`	**Yours, from P2** (or the reference, §4.8)
`ToolRegistry` — `include/tools/tool_registry.h`, `src/tools/tool_registry.cpp`	**You implement — the core of P3**
`Workspace::resolve` — `src/tools/workspace.cpp`	**You implement**
`calc_evaluate` — `src/tools/calculator_eval.cpp`	**You implement**
`JsonValue` — `include/json/json_value.h`	Provided
`Tool`, `ToolResult`, `ToolContext` — `include/tools/tool.h`	Provided
`Workspace` constructor, `root()`, `check_symlinks` — `src/tools/workspace_provided.cpp`	Provided
`CalcToken`, `calc_tokenize` — `include/tools/calculator.h`, `src/tools/calculator_tokenize.cpp`	Provided
`CalculatorTool`, `ReadFileTool`, `WriteFileTool`, `ListDirTool`	Provided
`ToolDispatcher`, `Harness`, transcripts, model clients, `main.cpp`	Provided

In the three files you implement (`tool_registry.h` / `.cpp`, `workspace.cpp`, `calculator_eval.cpp`):

- **Allowed:** `std::vector`, `std::list`, `std::forward_list`, raw arrays, `std::string`, `std::string_view`, `std::unique_ptr`, `std::function`, `std::filesystem`, `std::stack`, and anything in `<cstdint>`, `<cstddef>`, `<stdexcept>`, `<utility>`.
- **Forbidden:** `std::unordered_map`, `std::map`, `std::set`, `std::unordered_set`, `std::multimap`, `std::multiset`, regex, and third-party libraries. A forbidden container in one of these files scores 0 for that rubric row.

Your **tests** may use anything.

4.1 Provided Types You Will Use (read this first)

```cpp
struct ToolResult {
    bool        ok = false;
    std::string content;       // shown to the model when ok
    std::string error;         // shown to the model when !ok
    static ToolResult success(std::string content);
    static ToolResult failure(std::string error);
};

struct ToolContext {                        // everything a tool may touch; no
globals
    explicit ToolContext(Workspace ws);
    Workspace   workspace;
    std::size_t max_read_bytes = 64 * 1024;
};

class Tool {
public:
    virtual ~Tool() = default;
    virtual std::string      name()        const = 0;   // e.g. "read_file" —
the registry key
    virtual std::string      description() const = 0;   // one sentence, shown
to the model
    virtual const JsonValue& schema()      const = 0;   // which arguments it
accepts (JSON)
    virtual ToolResult invoke(const JsonValue& args, ToolContext& ctx) = 0;
// run it
};
```

`Tool` is **polymorphism** at work. The dispatcher holds a `Tool*` and calls `invoke` without knowing whether it is a calculator or a file reader. Adding a new tool means writing one class that inherits from `Tool` and registering it. Nothing else changes.

`JsonValue` holds one JSON value. You will only need it in your tests, to give a test tool a `schema()`: `JsonValue::object()` makes an empty object `{}`. Appendix B explains the rest.

4.2 Errors as Values — the Design Argument

In an agent loop, a tool failing is **normal**: the file doesn't exist, the path is forbidden, the expression divides by zero. The model is expected to read the error and try something else. So tools report failure by returning `ToolResult::failure("...")`, a plain value, instead of throwing an exception. Your `resolve` and `calc_evaluate` follow the same rule: they return `false` and fill in a message.

Exceptions are reserved for **programmer errors**, meaning bugs in C++ code (for example, calling `as_string()` on a JSON number). As a last line of defense, the dispatcher also catches anything a tool throws.

In your design log, explain in your own words why this split is sensible.

4.3 `ToolRegistry` — You Implement (the core of P3)

What it is

A **hash table** that maps a tool's name (a string) to the tool object. The dispatcher calls `find` once per tool call, so lookup should take O(1) time on average.

```
class ToolRegistry {
public:
    ToolRegistry();
    ~ToolRegistry();
    ToolRegistry(const ToolRegistry&) = delete;
    ToolRegistry& operator=(const ToolRegistry&) = delete;

    bool add(std::unique_ptr<Tool> tool);
    Tool* find(std::string_view name) const;
    std::size_t size() const noexcept;
    void for_each(const std::function<void(const Tool&)>& fn) const;

    // ---- Instrumentation (for tests and your design log) ----
    std::size_t bucket_count() const noexcept;
    double      load_factor() const noexcept;
    std::size_t max_probe_length() const;
    static std::size_t hash(std::string_view key) noexcept;

private:
    // ===== YOUR REPRESENTATION GOES HERE =====
};
```

Copying is deleted because the registry *owns* its tools. Two registries owning the same tools would both delete them — the shallow-copy problem from the special-member-functions lecture.

Function by function

Function	Purpose and who calls it	Exact behavior
`ToolRegistry()`	creates an empty table (called by `main.cpp`)	`size() == 0`; `bucket_count()` between 1 and 64 (a power of two is recommended)
`~ToolRegistry()`	frees everything	Every tool is destroyed. If your buckets hold `std::unique_ptr<Tool>`, this happens automatically.
`bool add(std::unique_ptr<Tool> tool)`	registers a tool (called by the provided `register_builtin_tools`)	If `tool` is `nullptr`, returns `false`. If a tool with the same `name()` is already registered, returns `false`, destroys the **new** tool, and keeps the original. Otherwise stores the tool (resizing first if needed — see below) and returns `true`. Amortized O(1).
`Tool* find(std::string_view name) const`	looks a tool up by name (called by the dispatcher for every tool call)	Returns a pointer to the registered tool, or `nullptr` if no tool has that name. Expected O(1). Does **not** transfer ownership: the caller must not delete it.
`std::size_t size() const noexcept`	number of registered tools	
`void for_each(fn) const`	calls `fn(tool)` exactly once for every registered tool (used by `--tools` and the system message, which sort the results themselves)	any order
`std::size_t bucket_count() const noexcept`	current number of buckets (slots) in your array	
`double load_factor() const noexcept`	`size() / bucket_count()` as a `double`	must be **≤ 1.0** whenever `add()` has returned
`std::size_t max_probe_length() const`	worst-case lookup cost, for measurement	**Chaining:** the number of entries in the longest chain. **Open addressing:** the largest number of slots

Function	Purpose and who calls it	Exact behavior
		examined to find any stored key, counting the key's home slot as 1. An empty table returns 0.
`static std::size_t hash(std::string_view key) noexcept`	your hash function, public so tests can measure it	must use every character of `key`

Design requirements

- **Collision strategy.** Choose **separate chaining** (each bucket is a linked list or a `std::vector`) or **open addressing with linear probing**, and justify the choice in your log. Appendix A is a primer. P3 never removes a tool, so open addressing needs no tombstones.
- **Hash function.** Use a real string hash. FNV-1a is given in Appendix A; djb2 is also fine. The hash must:
  - use **every** character
  - be **order-sensitive**: `"ab"` and `"ba"` must hash differently, so a plain sum of characters is not enough
  - return the **full-width** value, not one already reduced modulo your table size. Reduce it to an index separately (next bullet).

  `s[0] + s.size()` is **not** acceptable: it sends `tool_1` … `tool_9999` to a handful of buckets. The hidden tests check that your hash gives at least 9,990 different values for those 10,000 names.
- **Index from hash.** `index = hash(key) % bucket_count()`, or `hash(key) & (bucket_count() - 1)` if the bucket count is a power of two.
- **Resizing.** When the next `add` would push the load factor above your threshold, first double the bucket count, then **re-insert every entry** into the new array, recomputing each index. Typical thresholds are 0.75 for open addressing and 0.75–1.0 for chaining. Justify yours. With open addressing the threshold **must be below 1**: a completely full table has no empty slot, so `find` on a missing name would loop forever.
- **Pointer stability.** A `Tool*` returned by `find()` must stay valid after later `add()` calls cause resizes. Store `std::unique_ptr<Tool>` in your buckets. Moving a `unique_ptr` during a rehash moves only the pointer; the `Tool` object itself stays where it is.
- **Probe bound.** After inserting the 10,000 names `tool_0` … `tool_9999`, `max_probe_length()` must be **≤ 64**. For reference:

Hash + strategy	Typical `max_probe_length()`
FNV-1a with chaining	about 6

Hash + strategy	Typical `max_probe_length()`
FNV-1a with linear probing at 0.75	about 30
`s[0] + s.size()`	about 9,000
djb2 with linear probing in a power-of-two table	about 380 — djb2 clusters badly on these names, so measure before choosing this combination

Sample usage (what the provided code does):

```
ToolRegistry reg;
reg.add(std::make_unique<CalculatorTool>());       // true
reg.add(std::make_unique<CalculatorTool>());       // false: duplicate name,
new one destroyed
Tool* t = reg.find("calculator");                  // non-null
Tool* u = reg.find("teleport");                    // nullptr
ToolResult r = t->invoke(args, ctx);

int count = 0;
reg.for_each([&](const Tool& tool) {               // a lambda: runs once per
tool
    std::printf("%s\n", tool.name().c_str());
    ++count;                                       // [&] lets it change
`count`
});
```

A test tool for scale testing. To insert 10,000 tools you need a tool class whose name you choose. The starter's `tests/p3/test_p3.cpp` already contains this one:

```
class NamedTool : public Tool {
public:
    explicit NamedTool(std::string n) : n_(std::move(n)),
schema_(JsonValue::object()) {}
    std::string name() const override { return n_; }
    std::string description() const override { return "test tool"; }
    const JsonValue& schema() const override { return schema_; }
    ToolResult invoke(const JsonValue&, ToolContext&) override { return
ToolResult::success(n_); }
private:
    std::string n_;
    JsonValue   schema_;   // an empty JSON object: {}
};
```

4.4 `Workspace::resolve` — You Implement (text only)

What it is

The file tools receive paths from the model, and the model's output must be treated as untrusted. `Workspace` makes sure a path such as `notes/todo.txt` refers to something **inside** the workspace folder, and can never reach `/etc/passwd` or your home directory.

```
class Workspace {
public:
    explicit Workspace(std::filesystem::path root);    // provided
    const std::filesystem::path& root() const noexcept; // provided

    // YOU IMPLEMENT. Maps a model-supplied, workspace-relative path to an
    // absolute path inside root(). The target need not exist yet.
    // On success: returns true and stores the path in *out.
    // On rejection: returns false and stores a non-empty explanation in
*reason.
    bool resolve(std::string_view user_path,
                 std::filesystem::path* out,
                 std::string* reason) const;

    // Provided. Call this *after* resolve() succeeds, before any file I/O.
    // Rejects paths that escape through symbolic links.
    bool check_symlinks(const std::filesystem::path& resolved,
                        std::string* reason) const;
private:
    std::filesystem::path root_;   // absolute; set by the constructor
};
```

You only handle the **text** of the path. Symbolic links ("shortcuts" that point somewhere else) need operating-system knowledge outside this course, so the provided `check_symlinks` handles them.

Algorithm

Follow these steps in order:

1. If `user_path` is empty → reject (reason: `"empty path"`).
2. If `user_path` contains a `'\0'` byte → reject. A `std::string_view` can hold a NUL character, but the operating system would silently cut the path off there.
3. Build `std::filesystem::path p{std::string(user_path)}`. If `p.is_absolute()` or `p.has_root_path()` → reject. Only workspace-relative paths are allowed, even if an absolute path happens to point inside root.

4. Compute `n = p.lexically_normal()`. This removes `.` components and duplicate slashes, and folds `x/..` pairs. For example, `a/../b` becomes `b`, `./notes//todo.txt` becomes `notes/todo.txt`, and `a/..` becomes `.`.
5. If the **first component** of `n` is `..` (check `*n.begin() == ".."`) → reject. After normalization, a leading `..` means the path climbs above root at some point. That includes paths like `notes/../../ws/notes/todo.txt` that come back inside afterwards; reject those too.
6. If `n` is `.`, set `*out = root()`. Otherwise set `*out = root() / n`. Return `true`.

Your `resolve` must not throw and must not touch the filesystem (no `exists`, `canonical`, `create_directories`, …). Paths are pure text here, which is why `resolve` works for files that don't exist yet.

Required results

Input	Result
`notes/todo.txt`	accept → `root/notes/todo.txt`
`./notes//todo.txt`	accept → `root/notes/todo.txt`
`.`	accept → `root`
`a/../b`	accept → `root/b`
`a/..`	accept → `root`
`new/dir/file.txt` (doesn't exist)	accept → `root/new/dir/file.txt`
`my..notes.txt`	accept → `root/my..notes.txt`
`""`	reject (step 1)
`"notes\0/x"` (contains NUL)	reject (step 2)
`/etc/passwd`	reject (step 3)
`..`	reject (step 5)
`../etc/passwd`	reject (step 5)
`a/../../b`	reject (step 5)
`notes/../../ws/notes/todo.txt` (root is `…/ws`)	reject (step 5)

Testing the NUL row. `ws.resolve("notes\0/x", ...)` does **not** work: the string literal is cut off at the NUL, and `resolve` receives just `"notes"`, which it correctly accepts. Build the string explicitly instead: `std::string("notes") + '\0' + "/x"`.

4.5 How a File Tool Uses the Workspace

Every provided file tool follows the same steps before touching a file. Here is the pattern from `ReadFileTool`:

```
const JsonValue* p = args.find("path");
if (p == nullptr || p->type() != JsonValue::Type::String)
    return ToolResult::failure("missing required argument: path");

fs::path real;
std::string why;
if (!ctx.workspace.resolve(p->as_string(), &real, &why)) return
ToolResult::failure(why);
if (!ctx.workspace.check_symlinks(real, &why))          return
ToolResult::failure(why);

// ... now it is safe to open `real` ...
```

4.6 `calc_evaluate` — You Implement (two stacks)

What it is

The `calculator` tool works in two steps. The **provided** `calc_tokenize` does all the character-by-character work. It turns the text into a list of tokens, and it also checks that the list is well-formed. **You** evaluate that list with the **two-stack algorithm**:

```
"(2+3)*4"  ──calc_tokenize──►  [ (  2  +  3  )  *  4 ]  ──calc_evaluate──►  20
            provided                                       YOU
```

```
// include/tools/calculator.h
struct CalcToken {
    enum class Kind { Number, Plus, Minus, Times, Divide, LParen, RParen };
    Kind   kind  = Kind::Number;
    double value = 0.0;          // used only when kind == Number (may be
negative)
};

// PROVIDED.
bool calc_tokenize(std::string_view expression, std::vector<CalcToken>* out,
std::string* error);

// YOU IMPLEMENT (src/tools/calculator_eval.cpp).
bool calc_evaluate(const std::vector<CalcToken>& tokens, double* out,
std::string* error);
```

What the tokenizer guarantees. Every token list you receive is well-formed:

- numbers and operators alternate
- parentheses are balanced
- the list is not empty
- it does not start or end with an operator

A minus sign written directly in front of a number where a number is expected (`-3` , `2*-3` , `(-1)`) is already folded into that number's `value` . So you only ever see the four **binary** operators.

The algorithm

Keep two stacks: `values` (numbers) and `ops` (operators and `(`). Go through the tokens left to right:

Token	What to do
number	push its `value` onto `values`
`(`	push it onto `ops`
`)`	**apply** operators from `ops` until the top is `(` , then pop the `(`
`+ - * /`	while the top of `ops` is an operator (not `(`) whose precedence is **greater than or equal to** this one's, **apply** it; then push this operator

When the tokens run out, **apply** every operator left on `ops` . The single number left on `values` is the answer.

Apply means: pop one operator; pop the right operand, then the left operand (in that order); compute `left op right` ; push the result. `*` and `/` have precedence 2; `+` and `-` have precedence 1. The "greater than **or equal**" rule is what makes `10-4-3` evaluate left to right as `(10-4)-3 = 3` .

Appendix C walks through a full example.

Exact behavior

- On success: return `true` and store the result in `*out` .
- If an apply step would divide by zero: return `false` and set `*error` to exactly `"division by zero"` .
- You may assume the tokens came from `calc_tokenize` . Your function must still **never crash**: if a stack is unexpectedly empty, return `false` with a message. Never call `top()` on an empty stack.

- **Use loops, not recursion.** The hidden tests include an expression with 20,000 `+` terms and one with 5,000 nested parentheses. The two-stack algorithm handles both easily; a recursive helper would overflow the call stack.
- The provided `CalculatorTool` formats your result and also rejects results that are too large (infinite), so you don't have to.

Expression	Result
`2 + 3 * 4`	`14`
`10 - 4 - 3`	`3`
`100 / 10 / 5`	`2`
`(2 + 3) * 4`	`20`
`2 * (3 + 4) - 5`	`9`
`-3 * -3`	`9`
`7 / 2`	`3.5`
`1 / (2 - 2)`	error: `division by zero`

4.7 Where Your Code Is Called

Read these provided files to see your code in real use:

Provided file	Uses
`src/harness/tool_dispatcher.cpp`	`ToolRegistry::find`
`src/tools/builtin_tools.cpp`	`ToolRegistry::add`, `for_each`
`src/tools/file_tools.cpp`	`Workspace::resolve`
`src/tools/calculator_tool.cpp`	`calc_tokenize` (provided) and `calc_evaluate` (yours)
`tests/p3/test_loop.cpp`	end-to-end tests of the tool loop (provided; they pass once your three pieces work)

4.8 Carrying P2 Forward

Copy four files from your P2 repository into the starter:

From P2	To P3
`include/core/conversation.h`, `include/core/sentinel_scanner.h`	`include/core/`
`src/conversation.cpp`, `src/sentinel_scanner.cpp`	`src/core/`

Do **not** copy `message.h`; P3 provides a new one with `Role::Tool`. Your P2 `Conversation` never looks inside a `Message`, so it needs no changes. The provided `tests/p2/test_p2.cpp` must still pass.

If your P2 didn't work, configure with `cmake -S . -B build -DUSE_P2_REFERENCE=ON` to build against the instructor's reference instead. You lose no P3 points for this, but say so at the top of your design log. CMake remembers the setting, so switch back with `-DUSE_P2_REFERENCE=OFF` if you later fix your own code.

---

5. Common Pitfalls

1. **A rehash that loses or moves tools.** When resizing, re-insert every entry using the **new** bucket count. Move the `unique_ptr`s, never the `Tool` objects.
2. **Hashing only part of the key.** `tool_1234` and `tool_1243` must usually land in different places. Use every character.
3. **Checking the load factor after inserting.** If you insert first and resize afterwards, `load_factor()` can briefly exceed your threshold, and with open addressing a full table has nowhere to put the new entry. Check *before* inserting.
4. **Searching the path string for `".."`.** That wrongly rejects legal names like `my..notes.txt`. Use `lexically_normal()` and look at the first component, as in §4.4.
5. **Popping operands in the wrong order.** The first value you pop is the **right** operand. Swapping them makes `10 - 4` come out as `-6`.
6. **Using `>` instead of `>=` for precedence.** Then `10 - 4 - 3` evaluates as `10 - (4 - 3) = 9`.
7. **Forgetting to pop the `(`** after a `)`. It stays on `ops` and breaks everything after it.

---

6. Deliverables & Testing

The repository must build with CMake, compile with **zero warnings** under `-Wall -Wextra -Wpedantic` (`-DMH_WERROR=ON` checks this), and run clean under AddressSanitizer + UndefinedBehaviorSanitizer (on by default).

Repository structure (★ = you write or modify; everything else is provided):

```
├── CMakeLists.txt · README.md · P3_spec.md
├── include/
│   ├── core/          message.h · conversation.h ★(P2) · sentinel_scanner.h ★(P2)
```

```
|   ├── json/          json_value.h
|   ├── tools/         tool.h · tool_registry.h ★ (private section only) ·
workspace.h
|   |                  calculator.h · calculator_tool.h · file_tools.h ·
builtin_tools.h
|   ├── harness/       harness.h · tool_dispatcher.h · transcript.h
|   ├── model/         model_client.h · scripted_client.h · replay_client.h
|   └── util/          number_format.h
├── src/
|   ├── core/          conversation.cpp ★(P2) · sentinel_scanner.cpp ★(P2) ·
README.md
|   ├── json/          json_value.cpp
|   ├── tools/         tool_registry.cpp ★ · workspace.cpp ★ ·
calculator_eval.cpp ★
|   |                  workspace_provided.cpp · calculator_tokenize.cpp ·
calculator_tool.cpp
|   |                  file_tools.cpp · builtin_tools.cpp
|   ├── harness/       harness.cpp · tool_dispatcher.cpp · transcript.cpp
|   ├── model/         model_client.cpp · scripted_client.cpp ·
replay_client.cpp
|   └── main.cpp
├── reference/p2/      instructor P2 reference (used only with -
DUSE_P2_REFERENCE=ON)
├── scripts/           *.script demo sessions
├── workspace/         default sandbox for the file tools (your demo runs may
change it)
├── tests/
|   ├── mini_test.h       TEST / CHECK / CHECK_EQ / REQUIRE / CHECK_THROWS
framework
|   ├── test_helpers.h    TempDir, write_text, args1, fake model/input/output
|   ├── p2/test_p2.cpp    P2 regression (provided)
|   ├── p3/test_loop.cpp  tool-loop tests (provided)
|   ├── p3/test_p3.cpp ★  your P3 suite
|   └── e2e/              golden CLI sessions with their own workspace/, run
by ctest
└── docs/design-log-p3.md ★
```

Build & run (copy your P2 files in first, §4.8):

```
cmake -S . -B build && cmake --build build -j
./build/test_p2 && ./build/test_loop && ./build/test_p3    # or: cd build &&
ctest --output-on-failure
./build/test_p3 reg_                                       # only tests whose
names start with reg_
```

```
./build/miniharness --tools
./build/miniharness --script scripts/tools_demo.script
```

Your test suite (`tests/p3/test_p3.cpp`): at least **15** test cases using `mini_test.h`, named `<area>_<what>` (e.g. `reg_rejects_duplicate`, `ws_rejects_dotdot`, `calc_left_to_right`). The five examples in the starter count. The suite must cover:

1. **Registry basics:** add/find; a duplicate rejected with the original kept (check that `find` still returns the *first* tool's pointer); `nullptr` rejected; `find` of an absent name; `size()` after each of these.
2. **Registry `for_each`:** after adding several tools, `for_each` visits each one exactly once (use a lambda as in the §4.3 sample).
3. **Registry at scale (required):** insert `tool_0` … `tool_9999` using `NamedTool`. Assert that:
   - every name can be found
   - a pointer obtained after the first insert is still the same after the last
   - `load_factor() <= 1.0` after every add
   - `max_probe_length() <= 64`

   Print `bucket_count()`, `load_factor()`, and `max_probe_length()`.
4. **Alternative hash (required):** compute the longest chain that the weak hash `s[0] + s.size()` *would* produce for the same 10,000 names at your final bucket count. Hash the names yourself into a `std::vector<int>` of counts; you don't need a second table. Print it next to your real numbers. (Comparing djb2 too is welcome, but with chaining it does about as well as FNV-1a. The weak hash is the one that shows the lesson.)
5. **Workspace:** every row of the table in §4.4. Check the return value, and check `*out` for accepted paths or that `*reason` is non-empty for rejected ones. Use `TempDir` from `tests/test_helpers.h` as the root.
6. **Calculator:** every row of the table in §4.6, plus at least two expressions of your own. Use the `eval` helper in the starter's test file, which runs the provided `calc_tokenize` and then your `calc_evaluate`.

Design log (`docs/design-log-p3.md`): 500–800 words in Markdown. A template is in the starter. Cover:

- **At the top:** whether you used your own P2 or the reference, and which tools (including AI assistants) you used and for what.
- **Hash table:** collision strategy, hash function, load-factor threshold, and resize policy. Put your 10,000-tool measurements next to the alternative hash's, and explain briefly why `add()` is amortized O(1).
- **Pointer stability:** why `find()` results survive a resize in your design.

- **Calculator:** a step-by-step trace of the two stacks for `2 * (3 + 4) - 5`, in your own words.
- **Errors as values:** the argument from §4.2 in your own words.
- **One thing you would design differently** in hindsight.

Oral check-in. You may be asked to explain any part of your code: why it is structured the way it is, and what would happen if a specific piece changed. "The AI wrote it" is not a valid answer.

Submission files:

- **github.txt:** the URL of your GitHub repository.
- **github.zip:** a ZIP of your entire repository as a backup.

---

7. Suggested Schedule

Days	Goal	Checkpoint
1	Read §2 and §4, build the starter, port P2	`./build/test_p2` passes
2–3	`ToolRegistry` without resizing: `add`, `find`, `size`, `for_each`	basic registry tests pass; `--tools` lists the four built-in tools
4–6	Resizing, instrumentation, the scale test, and the alternative-hash measurement	10,000-tool test passes; numbers recorded
7	`Workspace::resolve` and its test table	every row of §4.4 passes
8–9	`calc_evaluate` and its tests	every row of §4.6 passes; `./build/test_loop` all green; the demo script matches §3
10	Integration: `ctest`, golden e2e sessions, sanitizers	everything green, zero warnings
11–12	Design log; final review of your code	submission files ready

---

8. Grading Rubric

Criterion	Points	Evaluation
Hash-table `ToolRegistry`	45	Hidden tests: empty table, add/find, duplicates, `nullptr`, ownership (tools destroyed exactly once), `for_each`, `load_factor`, 10,000 inserts with load factor ≤ 1.0, `max_probe_length()` ≤ 64, pointer stability across resizes, real string hash.
`Workspace::resolve`	15	Hidden version of the §4.4 table, checked directly and through the provided file tools. Also checks that `resolve` doesn't touch the filesystem.
`calc_evaluate`	10	Hidden tests: precedence, left-to-right order, parentheses, negative numbers, decimals, division by zero, very long expressions.
Integration	10	P2 regression suite (3), provided `test_loop` tests (5), and the `tests/e2e` golden sessions including `--tools` (2).
Test suite quality	10	Manual review against §6 items 1–6.
Design log	10	Quality of the arguments, measurements, and calculator trace.

Every automated test runs in its own process under ASan + UBSan. A crash, leak, or sanitizer report fails that test. A build with warnings loses up to 5 points.

Appendix A — Primer: Hash Tables

A hash table stores entries in an array of **buckets**. To find where a key goes, compute `index = hash(key) mod bucket_count`. If the bucket count is a power of two, `hash & (bucket_count - 1)` gives the same result and is cheaper. Two keys landing in the same bucket is a **collision**. There are two standard ways to handle collisions:

- **Separate chaining.** Each bucket holds a small list (a linked list or a `std::vector`) of entries. `find` computes the index and searches that one list. Simple, and it degrades gracefully as the table fills.

  ```
  bucket 0: [ "read_file" ]
  bucket 1: [ ]
  bucket 2: [ "calculator", "list_dir" ]   ← collision: both hashed to 2
  bucket 3: [ "write_file" ]
  ```

- **Open addressing (linear probing).** Each bucket holds at most one entry. On a collision, try the next slot (`i+1`, `i+2`, … wrapping around) until you find a free one. `find` follows the

same path until it finds the key or reaches an empty slot. Cache-friendly, but it slows down sharply as the load factor approaches 1.

The **load factor** α = size / bucket_count. With a good hash, the expected chain length (or probe count) depends on α, not on how many entries there are. That is why lookup is O(1) *expected*, and why you must **resize**: when α would cross your threshold, allocate twice as many buckets and re-insert every entry, because each key's index changes when the bucket count changes.

Why `add()` is amortized O(1). It is the same argument you made for `Conversation` in P2. A resize from *n* to *2n* buckets costs O(n), but it happens only after roughly *n/2* cheap inserts since the previous resize. Spread over those inserts, each one pays O(1) extra.

FNV-1a (64-bit) is a good, simple string hash:

```
std::uint64_t h = 14695981039346656037ULL;                    // "offset basis"
for (unsigned char c : key) { h ^= c; h *= 1099511628211ULL; }   // "FNV prime"
return static_cast<std::size_t>(h);
```

Appendix B — Primer: JSON

What it is. JSON (JavaScript Object Notation) is a plain-text format for structured data. Almost every web service and every LLM API uses it. It has exactly six kinds of value:

Kind	Looks like	In `JsonValue`
null	`null`	`Type::Null`
boolean	`true` , `false`	`Type::Bool` , read with `as_bool()`
number	`42` , `-2.5` , `1e3`	`Type::Number` , read with `as_number()` (a `double`)
string	`"hello"` — always double quotes	`Type::String` , read with `as_string()`
array	`[1, "two", true]` — an ordered list of values	`Type::Array` , `size()` and `[i]`
object	`{"path": "a.txt", "max_bytes": 100}` — a list of `"key": value` pairs	`Type::Object` , `find("key")`

Values can contain other values: an object member can be an array, and an array element can be an object. Inside a string, a few characters are written with a backslash: `\"` (quote), `\\` (backslash), `\n` (newline), `\t` (tab).

A tool call, piece by piece.

```
{"id":"c1","name":"read_file","arguments":
{"path":"notes/todo.txt","max_bytes":4096}}
```

- The outer `{ … }` is an **object** with three members.
- `"id":"c1"` — a string the model chooses; the harness echoes it back in the result so the model can match results to calls.
- `"name":"read_file"` — which tool to run. This is the key your `ToolRegistry::find` looks up.
- `"arguments":{ … }` — another object, holding the tool's arguments: a string `path` and a number `max_bytes`.

A tool result is built the same way by the dispatcher, using `JsonValue::dump()`:

```
{"id":"c1","ok":true,"content":"- buy milk\n"}       ← success
{"id":"c1","ok":false,"error":"no such file: x.txt"}  ← failure
```

Using `JsonValue` (all provided). You never parse JSON yourself. When you do need it, this is all there is to it:

```
// Reading — what every provided tool does with its arguments:
const JsonValue* p = args.find("path");                 // nullptr if there is
no "path"
if (p == nullptr || p->type() != JsonValue::Type::String)
    return ToolResult::failure("missing required argument: path");
std::string path = p->as_string();

// Building — e.g. an empty schema for a test tool, or a small object:
JsonValue empty = JsonValue::object();                  // {}
JsonValue obj   = JsonValue::object();
obj.set("expression", JsonValue::string("1+1"));        //
{"expression":"1+1"}
std::string text = obj.dump();                          // back to text
```

Calling the wrong accessor (`as_string()` on a number) throws `std::logic_error`, because that is a programmer error. Always check `type()` first. How `JsonValue` stores nested values internally is not part of this project.

Appendix C — Primer: Two-Stack Evaluation, Worked Example

Expression: `2 * (3 + 4) - 5`. Tokens: `2 * ( 3 + 4 ) - 5`.

Token	Action	`values` (bottom → top)	`ops` (bottom → top)
`2`	push value	`2`	
`*`	`ops` empty → push	`2`	`*`
`(`	push	`2`	`* (`
`3`	push value	`2 3`	`* (`
`+`	top is `(`, not an operator → push	`2 3`	`* ( +`
`4`	push value	`2 3 4`	`* ( +`
`)`	apply `+` (3+4=7); top is now `(` → pop it	`2 7`	`*`
`-`	top `*` has precedence 2 ≥ 1 → apply (2×7=14); `ops` empty → push `-`	`14`	`-`
`5`	push value	`14 5`	`-`
end	apply `-` (14−5=9)	`9`	

The answer is the single value left: **9**. Notice what the `>=` rule did when `-` arrived. The `*` already waiting on the stack was applied first, which is exactly "multiplication before subtraction". Also notice that `)` cleared the operators inside the parentheses before anything outside them could use the result.

Appendix D — Transcript and Script Formats

(Reference only — produced and consumed by provided code.)

Transcripts and scripts use the P2 block format, with one new role, `role: tool`. Tool results are single-line JSON (newlines inside strings are written `\n`), so they never collide with the `---` separator.

```
role: user
what is 6*7
---
role: assistant
Let me check.<|tool_call|>{"id":"c1","name":"calculator","arguments":
{"expression":"6*7"}}<|/tool_call|>
---
role: tool
<|tool_result|>{"id":"c1","ok":true,"content":"42"}<|/tool_result|>
---
role: assistant
```

```
6 times 7 is 42.<|end_conversation|>
---
```

Tool call: `<|tool_call|>` + a JSON object + `<|/tool_call|>`. `"name"` is required. `"id"` is echoed back in the result. `"arguments"` must be an object and may be left out (it then defaults to `{}`). A reply may contain several calls; at most 16 are run per reply.

Tool result: `<|tool_result|>` + `{"id":…,"ok":true,"content":…}` or `{"id":…,"ok":false,"error":…}` + `<|/tool_result|>`.